8 Deadlock

No, you can't always get what you want
You can't always get what you want
You can't always get what you want
But if you try sometimes you find
You get what you need

The philosophers Jagger & Richards

Deadlock is defined as when a system cannot make any forward progress. We define a system for the rest of the chapter as a set of rules by which a set of processes can move from one state to another, where a state is either working or waiting for a particular resource. Forward progress is defined as if there is at least one process working or we can award a process waiting for a resource that resource. In a lot of systems, deadlock is avoided by ignoring the entire concept [4, P.237]. Have you heard about turn it on and off again? For products where the stakes are low (User Operating Systems, Phones), it may be more efficient to allow deadlock. But in the cases where "failure is not an option" - Apollo 13, you need a system that tracks, breaks, or prevents deadlocks. Apollo 13 didn't fail because of deadlock, but it wouldn't be good to restart the system on liftoff.

Mission-critical operating systems need this guarantee formally because playing the odds with people's lives isn't a good idea. Okay so how do we do this? We model the problem. Even though it is a common statistical phrase that all models are wrong, the more accurate the model is to the system the higher the chance the method will work.

8.1 Resource Allocation Graphs

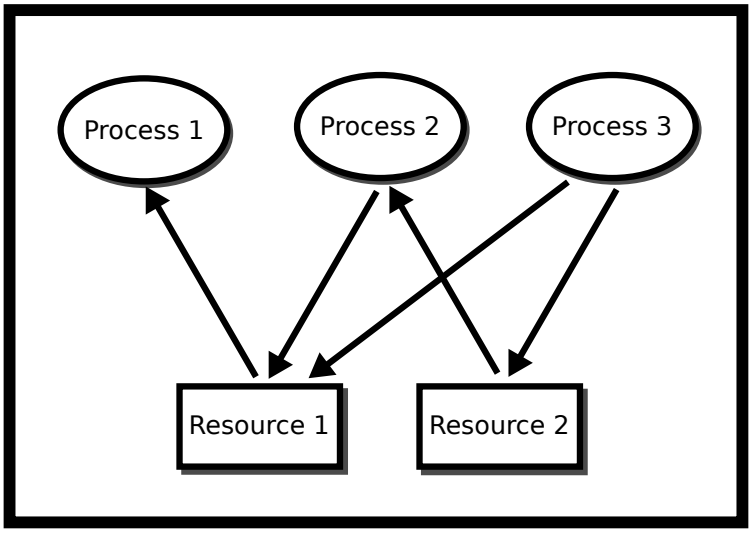


Figure 8.1: Resource allocation graph

One such way is modeling the system with a resource allocation graph (RAG). A resource allocation graph tracks which resource is held by which process and which process is waiting for a resource of a particular type. It is a simple yet powerful tool to illustrate how interacting processes can deadlock. If a process is *using* a resource, an arrow is drawn from the resource node to the process node. If a process is *requesting* a resource, an arrow is drawn from the process node to the resource node. If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will deadlock. For example, if process 1 holds resource A, process 2 holds resource B and process 1 is waiting for B and process 2 is waiting for A, then processes 1 and 2 will be deadlocked (see Figure 8.2). We'll make the distinction that the system is in deadlock by definition if all workers cannot perform an operation other than waiting. So, as long as every resource offers a single instance, detecting deadlock means searching the graph for a cycle. Both the processes and the resources are nodes, and every edge has a direction, so this is cycle detection in a *directed* graph. The usual tool is a depth-first search that records two different facts about each node: whether we have finished exploring it, and whether it sits on the path we are currently walking. The pseudocode below keeps each of those facts in a `set`, which you can picture as a hash set of node identifiers offering constant time insertion, removal and membership tests.

```
// Pseudocode. A node is either a process or a resource.
// finished: nodes that have already been explored completely.
// path:     nodes on the current search stack, i.e. how we got here.

static bool visit(const graph *g, node n, set *finished, set *path);

bool is_cyclic(const graph *g) {
    set *finished = set_create();
    set *path = set_create();
    bool cyclic = false;
```

```
    // A search from one node only explores what that node reaches, so
    // every node gets a turn as the starting point. 'finished' is
         shared
    // across those searches, so no node is explored twice.
    for (each node n in g) {
        if (visit(g, n, finished, path)) {
            cyclic = true;
            break;
        }
    }

    // Note the single exit: returning as soon as a cycle turns up
    // would skip the set_destroy calls below and leak both sets.
    set_destroy(finished);
    set_destroy(path);
    return cyclic;
}

static bool visit(const graph *g, node n, set *finished, set *path) {
    // n is on the route that brought us here: we walked in a circle
    if (set_contains(path, n)) return true;
    // n was explored before, and no cycle was found through it
    if (set_contains(finished, n)) return false;

    set_add(path, n);
    for (each node m that n points to) {
        if (visit(g, m, finished, path)) return true;
    }
    set_remove(path, n); // n is behind us; no longer on the path
    set_add(finished, n);

    return false;
}
```

Every node is expanded at most once, so the search costs $O(V + E)$ for V nodes and E edges.

Those two sets do different jobs, and collapsing them into one is a popular way to get this wrong. The tempting shortcut is "if I have seen this node before, there is a cycle", and it is broken on *any* graph rather than merely unsuited to directed ones. On an undirected graph it fires on the very first edge you walk, because the node you just came from has already been seen. On a directed graph it survives that test and fails in a quieter way. Suppose processes 1 and 2 are both waiting for resource A, which process 3 currently holds. The edges are $P_1 \to A$, $P_2 \to A$ and $A \to P_3$. Nothing is deadlocked here – process 3 will finish and release A – but a search that starts at process 1 explores A and process 3, and when the outer loop restarts at process 2 it reaches A a second time. Under the shortcut that second sighting is a deadlock report for a system that is running perfectly well, and the same false alarm happens inside a single search whenever two routes converge on one node. Seeing a node twice only means two paths lead to it; it is a cycle only when the second sighting happens while the first visit is still unfinished, which is precisely what `path` records.

Remember that the answer this search gives you is only as strong as the single instance assumption. If a resource in the cycle offers several interchangeable instances, a process outside the cycle may still return an instance and free somebody to continue, so a cycle remains necessary for deadlock but stops being sufficient.

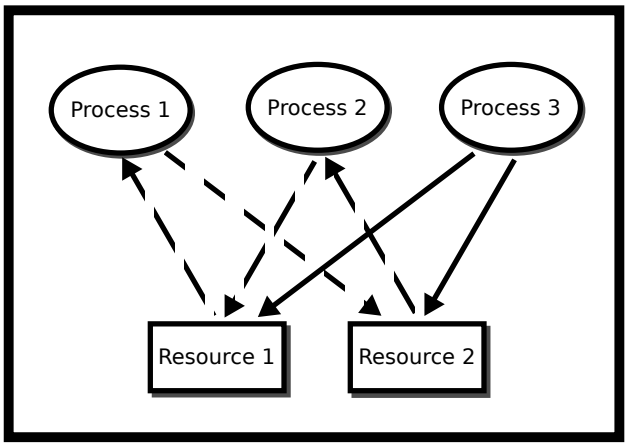


Figure 8.2: Graph based Deadlock

In this graph, process 1 holds resource 1 and is requesting resource 2, while process 2 holds resource 2 and is requesting resource 1. The dashed edges mark that cycle, so processes 1 and 2 are deadlocked, and process 3, which is requesting both resources, is stuck behind them.

8.2 Coffman Conditions

Surely cycles in RAGs happen all the time in an OS, so why doesn't it grind to a halt? You may not see deadlock because the OS may **preempt** some processes breaking the cycle but there is still a chance that your three lonely processes could deadlock.

There are four conditions for deadlock, and they are *necessary*: if a system is deadlocked, all four hold, so a system that breaks any one of them cannot deadlock. In general they are not *sufficient*: all four can hold and the system can still make progress, for example when a resource has several interchangeable instances and a process outside the cycle returns one. If every resource has a single instance, the conditions are sufficient as well, so a circular wait is a deadlock. These are known as the Coffman Conditions [1].

- Mutual Exclusion: No two processes can obtain a resource at the same time.
- Circular Wait: There exists a cycle in the Resource Allocation Graph, or there exists a set of processes {P1, P2,…} such that P1 is waiting for resources held by P2, which is waiting for P3,…, which is waiting for P1.
- Hold and Wait: Once a resource is obtained, a process keeps the resource locked.
- No pre-emption: Nothing can force the process to give up a resource.

Proof: Suppose every resource has a single instance and a process waits for at most one resource at a time. Call a set of processes *D deadlocked* if every process in *D* is waiting for a resource held by another process in *D*. When *D* is every process, this is the whole-system deadlock from the start of the chapter. We

show two things: if a set of processes is deadlocked, all four Coffman conditions hold; and if the system has mutual exclusion, hold and wait, and no pre-emption, then any cycle in the resource allocation graph is a deadlock.

→ Suppose D is deadlocked.

- Mutual exclusion: a process in D is waiting for a resource that someone else holds. If that resource could be shared, it would simply be given to the waiting process, which would then not be waiting.
- Hold and wait: take any $p \in D$ waiting for resource r. The holder q of r is also in D, so q is itself waiting – while holding r.
- No pre-emption: if the system could take r away from q, it could give r to p, and p would not be stuck.
- Circular wait: start at any $p_0 \in D$. Let p_1 be the holder of the resource p_0 waits for, p_2 the holder of the resource p_1 waits for, and so on. Every p_i is in D, so this chain never stops. But D is finite, so some process eventually repeats: $p_i = p_j$ for some $i < j$. Then $p_i \to p_{i+1} \to \cdots \to p_{j-1} \to p_i$ is a cycle in the resource allocation graph.

← Suppose the system has mutual exclusion, hold and wait, and no pre-emption, and the resource allocation graph contains a cycle $p_1 \to r_1 \to p_2 \to r_2 \to \cdots \to p_k \to r_k \to p_1$. That is, p_i is waiting for r_i, which is held by p_{i+1}, and p_{k+1} means p_1. For contradiction, suppose some process on the cycle eventually gets the resource it is waiting for, and let p_i be the *first* one to do so, at time t. Resource r_i has a single instance and cannot be shared, so p_{i+1} must have given it up before time t. There is no pre-emption, so nobody took r_i away from p_{i+1}; p_{i+1} released it itself. Because of hold and wait, p_{i+1} does not release anything while it is still waiting, so p_{i+1} must have been granted r_{i+1} before time t. That contradicts p_i being the first. So no process on the cycle ever gets its resource: the processes on the cycle are deadlocked. □

The only place the proof used single instances is the ← direction: if r_i had a second copy, a process outside the cycle could release that copy and let p_i continue. That is why, with multi-instance resources, a cycle is necessary for deadlock but not sufficient.

If a system breaks any of them, it cannot have deadlock! Consider the scenario where two students need to write both pen and paper and there is only one of each. Breaking mutual exclusion means that the students share the pen and paper. Breaking circular wait could be that the students agree to grab the pen then the paper. As proof by contradiction, say that deadlock occurs under the rule and the conditions. Without loss of generality, that means a student would have to be waiting on the pen while holding the paper and the other waiting on the paper while holding the pen. We have contradicted ourselves because one student grabbed the paper without grabbing the pen, so deadlock fails to occur. Breaking hold and wait could be that the students try to get the pen and then the paper and if a student fails to grab the paper then they release the pen. This introduces a new problem called *livelock* which will be discussed later. Breaking preemption means that if the two students are in deadlock the teacher can come in and break up the deadlock by giving one of the students a held item or telling both students to put the items down.

Livelock relates to deadlock. Consider the breaking hold-and-wait solution as above. Though deadlock is avoided, consider a variant of this solution where the two students reach for the items in opposite orders: one reaches for the pen first and the other reaches for the paper first. Each grabs their first item, fails to get the second, puts the first down, and tries again. If they keep doing this in lockstep, both are busy but no work will be done. Livelock is generally harder to detect because the processes generally look like they are working to the outside operating system whereas in deadlock the operating system generally knows when two processes

are waiting on a system-wide resource. Another problem is that there are necessary conditions for livelock (i.e. deadlock fails to occur) but not sufficient conditions – meaning there is no set of rules where livelock has to occur. You must prove that a particular system is free of livelock, usually with what is known as an invariant. One has to enumerate each of the steps of a system and if each of the steps eventually – after some finite number of steps – leads to forward progress, the system fails to livelock. There are even better systems that prove bounded waits; a system can only be livelocked for at most n cycles which may be important for something like stock exchanges.

8.3 Approaches to Solving Livelock and Deadlock

Ignoring deadlock is the most obvious approach. Quite humorously, the name for this approach is called the ostrich algorithm. Though there is no apparent source, the idea for the algorithm comes from the concept of an ostrich sticking its head in the sand. When the operating system detects deadlock, it does nothing out of the ordinary, and any deadlock usually goes away. An operating system preempts processes when stopping them for context switches. The operating system can interrupt any system call, potentially breaking a deadlock scenario. The OS also makes some files read-only thus making the resource shareable. The ostrich algorithm accepts that a malicious or badly written program can still deadlock; the operating system simply doesn't try to prevent it. For everyday life, this tends to be fine. When it is not we can turn to the following method.

Deadlock detection allows the system to enter a deadlocked state. After entering, the system uses the information to break deadlock. As an example, consider multiple processes accessing files. The operating system can keep track of all of the files/resources through file descriptors at some level either abstracted through an API or directly. If the operating system detects a directed cycle in the operating system file descriptor table it may break one process' hold through scheduling for example and let the system proceed. Why this is a popular choice in this realm is that there is no way of knowing which resources a program will select without running the program. This is an extension of Rice's theorem [3] that says that we cannot know any semantic feature without running the program (semantic meaning like what files it tries to open). So theoretically, it is sound. The problem then gets introduced that we could reach a livelock scenario if we preempt a set of resources again and again. The way around this is mostly probabilistic. The operating system chooses a random resource to break `hold-and-wait`. Now even though a user can craft a program where breaking hold and wait on each resource will result in a livelock, this doesn't happen as often on machines that run programs in practice or the livelock that does happen happens for a couple of cycles. These systems are good for products that need to maintain a non-deadlocked state but can tolerate a small chance of livelock for a short time.

In addition, we have the *Banker's Algorithm*, whose basic premise is that the bank never runs dry, which prevents livelock. Feel free to check out the appendix for more details.

8.4 Dining Philosophers

The Dining Philosophers problem is a classic synchronization problem. Imagine we invite n (let's say 6) philosophers to a meal. We will sit them at a table with 6 chopsticks, one between each philosopher. A philosopher alternates between wanting to eat or think. To eat the philosopher must pick up the two chopsticks on either side of their position. Dijkstra's original version of the problem used forks and a bowl of spaghetti,

and a skeptic might point out that you can eat spaghetti with one fork, so many retellings use chopsticks instead – nobody eats with one chopstick. The code and figures in this chapter still say fork; the two words mean the same thing. Each chopstick is shared with a neighbor.

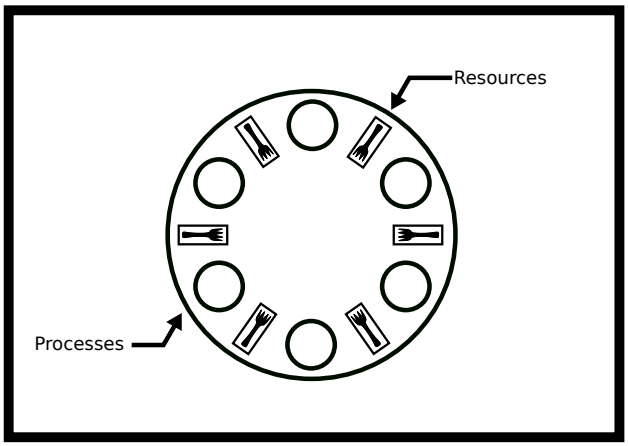


Figure 8.3: Dining Philosophers

Is it possible to design an efficient solution such that all philosophers get to eat? Or, will some philosophers starve, never obtaining a second chopstick? Or will all of them deadlock? For example, imagine each guest picks up the chopstick on their left and then waits for the chopstick on their right to be free. Oops - our philosophers have deadlocked! Each philosopher is essentially the same, meaning that each philosopher has the same instruction set based on the other philosopher i.e. you can't tell every even philosopher to do one thing and every odd philosopher to do another thing.

8.4.1 Failed Solutions

```
void* philosopher(void* forks){
  info phil_info = forks;
  pthread_mutex_t* left_fork = phil_info->left_fork;
  pthread_mutex_t* right_fork = phil_info->right_fork;
  while(phil_info->simulation){
    pthread_mutex_lock(left_fork);
    pthread_mutex_lock(right_fork);
    eat(left_fork, right_fork);
    pthread_mutex_unlock(left_fork);
    pthread_mutex_unlock(right_fork);
  }
}
```

This looks good, but what if everyone picks up their left fork and then waits for their right fork? We have deadlocked the program. It is important to note that deadlock doesn't happen all the time and the probability

that this solution deadlocks goes down as the number of philosophers goes up. What is important to note is that eventually this solution will deadlock, letting threads starve which is bad. Here is a simple resource allocation graph that shows how the system could be deadlocked.

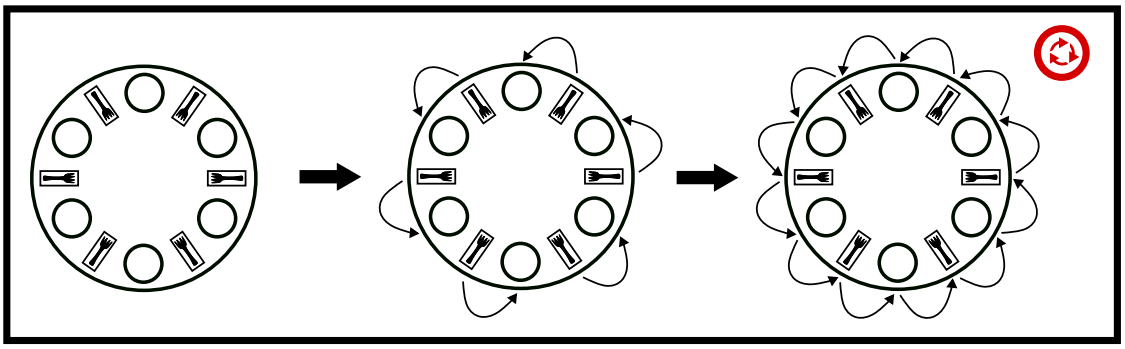


Figure 8.4: Left right dining philosopher cycle

So now you are thinking about breaking one of the Coffman Conditions. Let's break Hold and Wait!

```
void* philosopher(void* forks){
  info phil_info = forks;
  pthread_mutex_t* left_fork = phil_info->left_fork;
  pthread_mutex_t* right_fork = phil_info->right_fork;
  while(phil_info->simulation){
    int left_succeed = pthread_mutex_trylock(left_fork);
    if (!left_succeed) {
      sleep();
      continue;
    }
    int right_succeed = pthread_mutex_trylock(right_fork);
    if (!right_succeed) {
      pthread_mutex_unlock(left_fork);
      sleep();
      continue;
    }
    eat(left_fork, right_fork);
    pthread_mutex_unlock(left_fork);
    pthread_mutex_unlock(right_fork);
  }
}
```

Now our philosopher picks up the left fork and tries to grab the right. If it's available, they eat. If it's not available, they put the left fork down and try again. No deadlock! But, there is a problem. What if all the philosophers pick up their left at the same time, try to grab their right, put their left down, pick up their left, try to grab their right and so on. Here is what a time evolution of the system would look like.

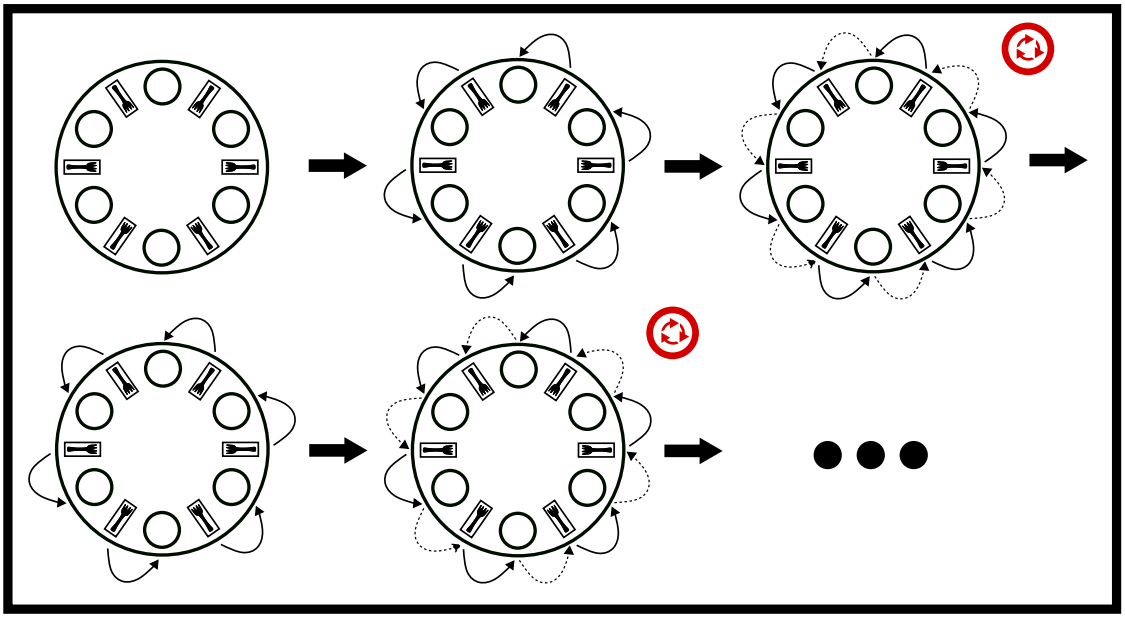


Figure 8.5: Livelock Failure

We have now livelocked our solution! Our poor philosophers are still starving, so let's give them some proper solutions.

8.5 Viable Solutions

The naive arbitrator solution has one arbitrator, a mutex for example. Have each of the philosophers ask the arbitrator for permission to eat or trylock an arbitrator mutex. This solution allows one philosopher to eat at a time. When they are done, another philosopher can ask for permission to eat. This prevents deadlock because there is no circular wait! No philosopher has to wait for any other philosopher. The advanced arbitrator solution is to implement a class that determines if the philosopher's forks are in the arbitrator's possession. If they are, they give them to the philosopher, let him eat, and take the forks back. This has the bonus of being able to have multiple philosophers eat at the same time.

There are a lot of problems with these solutions. One is that they are slow and have a single point of failure. Assuming that all the philosophers are good-willed, the arbitrator needs to be fair. In practical systems, the arbitrator tends to give forks to the same processes because of scheduling or pseudo-randomness. Another important thing to note is that this prevents deadlock for the entire system. But in our model of dining philosophers, the philosopher has to release the lock themselves. Then, you can consider the case where a malicious philosopher (let's say Descartes because of his Evil Demons) could hold on to the arbitrator forever. He would make forward progress and the system would make forward progress but there is no way of ensuring that each process makes forward progress without assuming something about the processes or having true preemption – meaning that a higher authority (let's say Steve Jobs) tells them to stop eating forcibly.

Proof: The arbitrator solution doesn't deadlock

The proof is about as simple as it gets. A philosopher only picks up forks while holding the arbitrator, and puts both forks down before letting go of it. So whoever holds the arbitrator finds every fork on the table and never waits for one. Everyone else is waiting only for the arbitrator, which is held by a philosopher who is not waiting. A cycle in the resource allocation graph needs every philosopher on it to be waiting, so no cycle can form, and without circular wait there is no deadlock, which is what we needed to show.

□

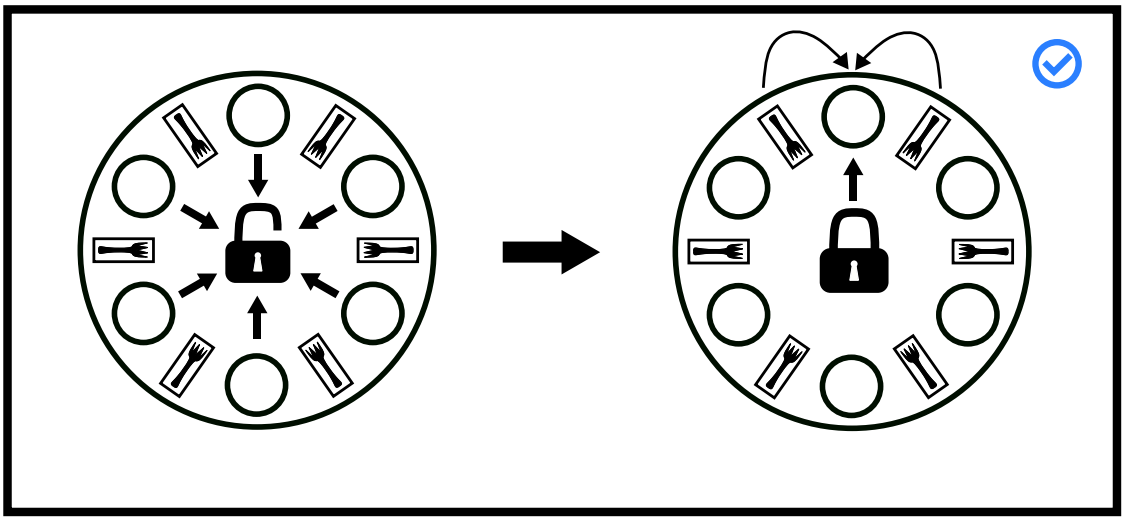


Figure 8.6: Arbitrator Diagram

8.5.1 Leaving the Table (Stallings' Solution)

Why does the first solution deadlock? Well, there are n philosophers and n chopsticks. What if there is only 1 philosopher at the table? Can we deadlock? No. How about 2 philosophers? 3? You can see where this is going. Stallings' [5, P. 280] solution removes philosophers from the table until deadlock is not possible – think about what the magic number of philosophers at the table is. The way to do this in the actual system is through semaphores and letting a certain number of philosophers through. This has the benefit that multiple philosophers can be eating.

In the case that the philosophers aren't evil, this solution requires a lot of time-consuming context switching. There is also no reliable way to know the number of resources beforehand. In the dining philosophers case, this is solved because everything is known but trying to specify an operating system where a system doesn't know which file is going to get opened by what process can lead to a faulty solution. And again since semaphores are system constructs, they obey system timing clocks which means that the same processes tend to get added back into the queue again. Now if a philosopher becomes evil, then the problem becomes that there is no preemption. A philosopher can eat for as long as they want and the system will continue to function but that means the fairness of this solution can be low in the worst case. This works best with timeouts or forced context switches to ensure bounded wait times.

Proof: Stallings' Solution Doesn't Deadlock. Let's number the philosophers $\{p_0, p_1, .., p_{n-1}\}$ and the resources $\{r_0, r_1, .., r_{n-1}\}$. A philosopher p_i needs resources r_i and $r_{(i+1) \bmod n}$. Without loss of generality, let us take p_i out of the picture. Each resource had exactly two philosophers that could use it. Now resources r_i and $r_{(i+1) \bmod n}$ each have only one philosopher that could use them. Even if hold and wait, no preemption, and mutual exclusion are present, the resources can never enter a state where one philosopher requests them and they are held by another philosopher because only one philosopher can request them. Since there is no way to generate a cycle otherwise, circular wait cannot hold. Since circular wait cannot hold, deadlock cannot happen. □

Here is a visualization of the worst-case. The system is about to deadlock, but the approach resolves it.

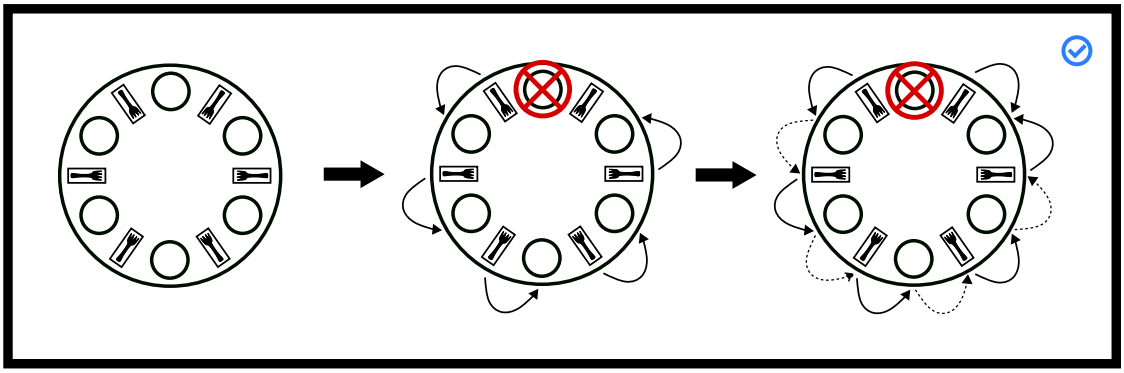


Figure 8.7: Stallings' solution almost deadlock

Because one seat is empty, there are more forks than philosophers. Even if all five remaining philosophers hold their left fork, one fork is still free, so at least one philosopher can pick up a second fork and eat.

8.5.2 Partial Ordering (Dijkstra's Solution)

This is Dijkstra's solution [2, P. 20]. He was the one to propose this problem on an exam. Why does the first solution deadlock? Dijkstra thought that the last philosopher who picks up his left fork (causing the solution to deadlock) should pick up his right. He accomplishes it by numbering the forks 1..n, and tells each of the philosophers to pick up his lower number fork. Let's run through the deadlock condition again. Everyone tries to pick up their lower number fork first. Philosopher 1 gets fork 1, Philosopher 2 gets fork 2, and so on until we get to Philosopher n. They have to choose between fork 1 and n. Fork 1 is already held up by philosopher 1, so they can't pick up that fork, meaning he won't pick up fork n. We have broken `circular wait`! Meaning deadlock isn't possible.

Some problems are that an entity either needs to know the finite set of resources in advance or be able to produce a consistent partial order such that circular wait cannot happen. This also implies that there needs to be some entity, either the operating system or another process, deciding on the number and all of the philosophers need to agree on the number as new resources come in. As we have also seen with previous solutions, this relies on context switching. This prioritizes philosophers that have already eaten but can be made fairer by introducing random sleeps and waits.

Proof: Dijkstra's Solution Doesn't Deadlock

The proof is similar to the previous proof. Let's number the philosophers $\{p_0, p_1, \ldots, p_{n-1}\}$ and the forks $\{r_0, r_1, \ldots, r_{n-1}\}$, where philosopher p_i needs forks r_i and $r_{(i+1) \bmod n}$. Everyone picks up their lower-numbered fork first. For p_0 through p_{n-2} that means r_i and then r_{i+1}. The last philosopher, p_{n-1}, needs r_{n-1} and r_0, so they pick up r_0 first and then r_{n-1} – the reverse of everyone else. Notice that p_0 and p_{n-1} both want r_0 as their *first* fork, and nobody picks up their second fork before their first. So whichever of the two does not hold r_0 is holding no fork at all. A philosopher holding nothing cannot be part of a cycle in the resource allocation graph, because everyone on a cycle holds the fork that the previous philosopher is waiting for. There are two cases.

If p_{n-1} does not hold r_0, then p_{n-1} holds nothing, and we can take p_{n-1} out of the picture.

If p_{n-1} does hold r_0, then p_0 holds nothing, and we can take p_0 out of the picture.

Either way, at most $n-1$ philosophers are sharing the n forks, which is exactly the situation in Stallings' proof above, so circular wait cannot happen. Since we can't reach deadlock in either case, this solution cannot deadlock, which is what we needed to show.

□

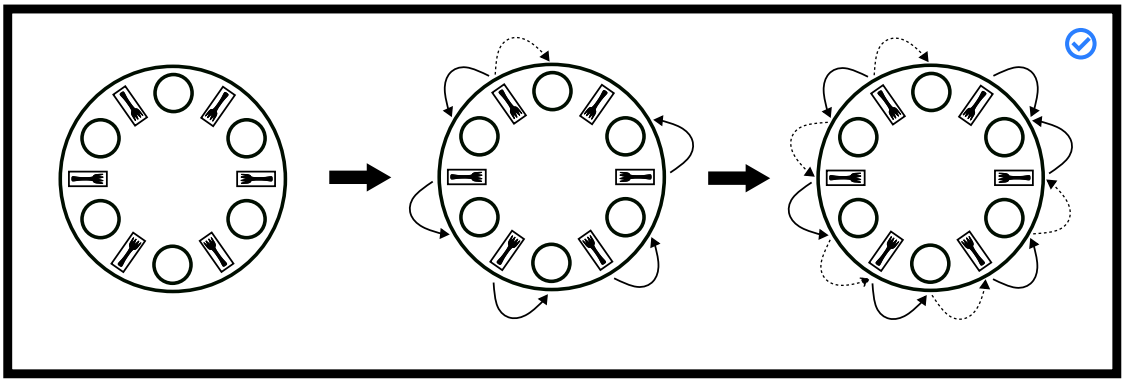


Figure 8.8: Dijkstra's partial ordering solution

There are a few other solutions (clean/dirty forks and the actor model) in the appendix.

8.6 Topics

- Coffman Conditions
- Resource Allocation Graphs
- Dining Philosophers
- Failed DP Solutions
- Livelocking DP Solutions
- Working DP Solutions: Benefits/Drawbacks
- Ron Swanson Deadlock

8.7 Questions

- What are the Coffman conditions?
- What does each of the Coffman conditions mean? Define each one.
- Give a real-life example of breaking each Coffman condition in turn. A situation to consider: Painters, Paint, Paint Brushes etc. How would you assure that work would get done?
- Which Coffman condition is unsatisfied in the following snippet?

```
// Get both locks or none
pthread_mutex_lock(a);
if(pthread_mutex_trylock( b )) { /* failure */
  pthread_mutex_unlock( a );
}
```

- The following calls are made

```
// Thread 1
pthread_mutex_lock(m1) // success
pthread_mutex_lock(m2) // blocks

// Thread 2
pthread_mutex_lock(m2) // success
pthread_mutex_lock(m1) // blocks
```

  What happens and why? What happens if a third thread calls `pthread_mutex_lock(m1)` ?

- How many processes are blocked? As usual, assume that a process can complete if it can acquire all of the resources listed below.
  - P1 acquires R1
  - P2 acquires R2
  - P1 acquires R3
  - P2 waits for R3
  - P3 acquires R5
  - P1 waits for R4
  - P3 waits for R1
  - P4 waits for R5
  - P5 waits for R1

Draw out the resource graph!

Bibliography

[1] Edward G Coffman, Melanie Elphick, and Arie Shoshani. System deadlocks. *ACM Computing Surveys (CSUR)*, 3(2):67–78, 1971.

[2] Edsger W. Dijkstra. Hierarchical ordering of sequential processes. Published in Acta Informatica 1(2):115–138, 1971, 1971. URL http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD310.PDF.

[3] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953. ISSN 00029947. URL http://www.jstor.org/stable/1990888.

[4] A. Silberschatz, P.B. Galvin, and G. Gagne. *OPERATING SYSTEM PRINCIPLES, 7TH ED*. Wiley student edition. Wiley India Pvt. Limited, 2006. ISBN 9788126509621. URL https://books.google.com/books?id=WjvXOHmVTlMC.

[5] William Stallings. *Operating Systems: Internals and Design Principles 7th Ed. by Stallings (International Economy Edition)*. PE, 2011. ISBN 9332518807. URL https://www.amazon.com/Operating-Systems-Internals-Principles-International/dp/9332518807?SubscriptionId=0JYN1NVW651KCA56C102&tag=techkie-20&linkCode=xm2&camp=2025&creative=165953&creativeASIN=9332518807.